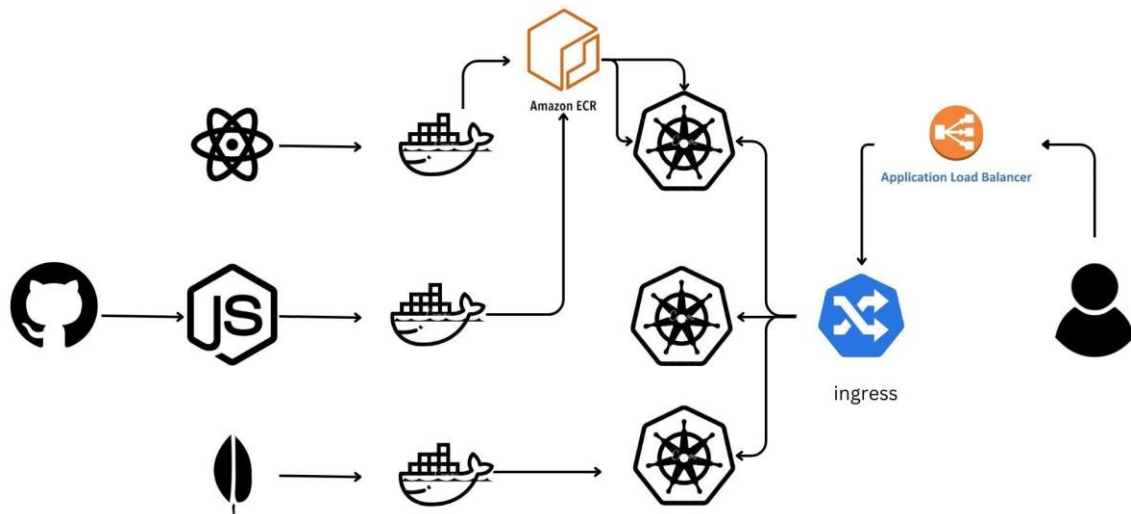


Three Tier Application Deployment on Kubernetes

1. Frontend: React.js
2. Backend: Node.js (JavaScript)
3. Database: MongoDB



STEP 1

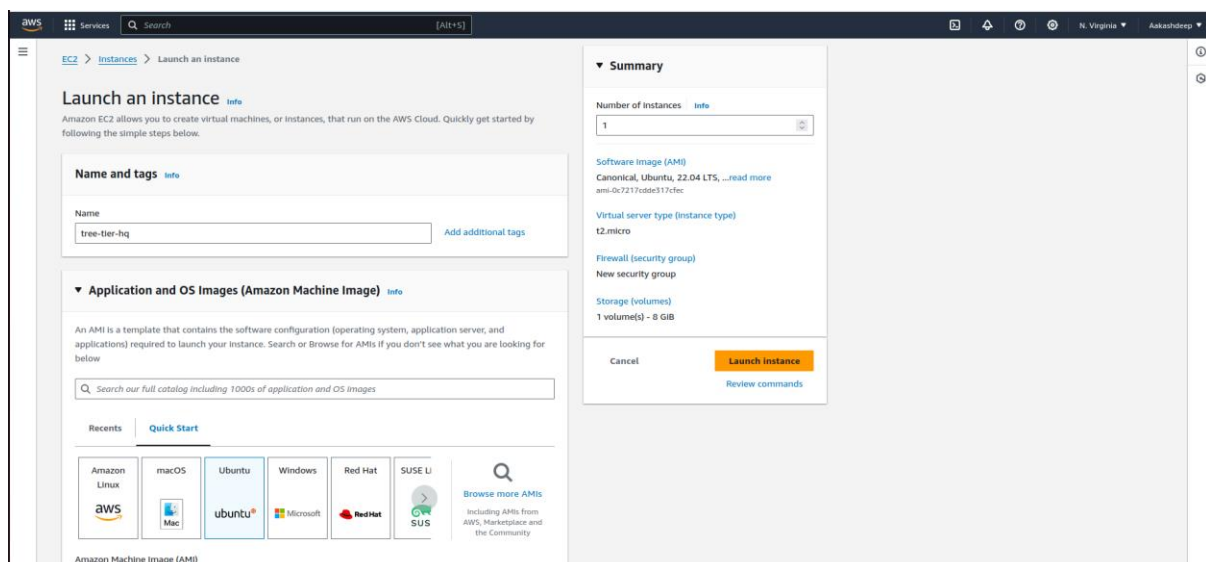
Setting up EC2 for Application Deployment

Sign in to the AWS Management Console

In the AWS Management Console, navigate to the "Services" dropdown and select "EC2".

In the EC2 Dashboard, click on the "Instances" in the left navigation pane.

Click the "Launch Instances" button.



Key pair (login) Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required
tree-tier-hq-key [Create new key pair](#)

Network settings Info [Edit](#)

Network Info
vpc-08af06a592d9a483d

Subnet Info
No preference (Default subnet in any availability zone)

Auto-assign public IP Info
Enable

Firewall (security groups) Info
A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group ☐ Select existing security group

We'll create a new security group called 'launch-wizard-12' with the following rules:

☒ Allow SSH traffic from
Helps you connect to your instance: Anywhere (0.0.0.0/0)

☐ Allow HTTPS traffic from the internet
To set up an endpoint, for example when creating a web server.

☐ Allow HTTP traffic from the internet
To set up an endpoint, for example when creating a web server.

Summary

Number of instances Info
1

Software image (AMI)
Canonical, Ubuntu, 22.04 LTS, ...read more
ami-0c7217cd317c1ec

Virtual server type (instance type)
t2.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Cancel [Launch Instance](#) [Review commands](#)

Click the **“launch instance”** button

View Instances:

Once the instance is launched, you can view it on the Instances page. It may take a few minutes for the instance to be in the running state.

Instances (1) Info

Find instance by attribute or tag (case-sensitive) Any state

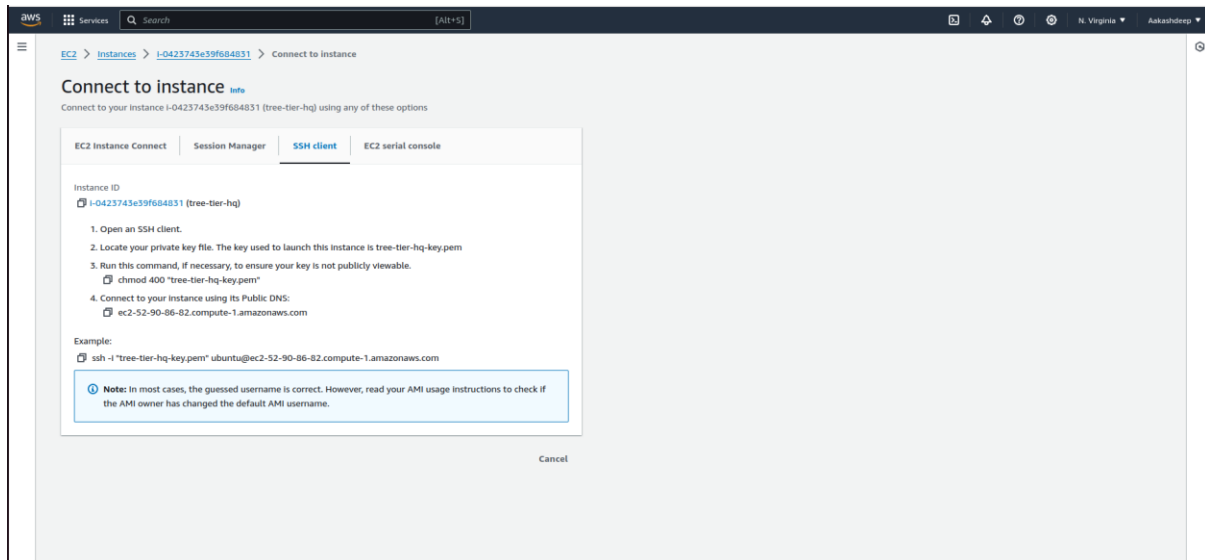
Instance state: running Clear filters

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elu
☐	tree-tier-hq	i-0425745e59f684831	Running	t2.micro	Initializing	View alarms +	us-east-1d	ec2-52-90-86-82.comp...	52.90.86.82	-

Select an instance

Connect using SSH:

Open a terminal on your local machine.



```
ubuntu@ip-172-31-82-137: ~  
aakashdeep@ubuntu:~/Downloads$ sudo ssh -i "tree-tier-hq-key.pem" ubuntu@ec2-52-90-86-82.compute-1.amazonaws.com  
[sudo] password for aakashdeep:  
The authenticity of host 'ec2-52-90-86-82.compute-1.amazonaws.com (52.90.86.82)' can't be established.  
ED25519 key fingerprint is SHA256:K99HpPWA1hPCMORDnHZtIYYu9tgtEVR1yQ36To30nEI.  
This key is not known by any other names  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added 'ec2-52-90-86-82.compute-1.amazonaws.com' (ED25519) to the list of known hosts.  
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 6.2.0-1017-aws x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/advantage  
  
System information as of Tue Jan 30 11:06:49 UTC 2024  
  
System load:  0.0      Processes:            97  
Usage of /:   20.5% of 7.57GB   Users logged in:     0  
Memory usage: 21%      IPv4 address for eth0: 172.31.82.137  
Swap usage:   0%  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
The list of available updates is more than a week old.  
To check for new updates run: sudo apt update  
  
The programs included with the Ubuntu system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.  
  
ubuntu@ip-172-31-82-137:~$ ls  
ubuntu@ip-172-31-82-137:~$
```

-- now connected to your EC2 instance via SSH

STEP 2

Log in to GitHub:

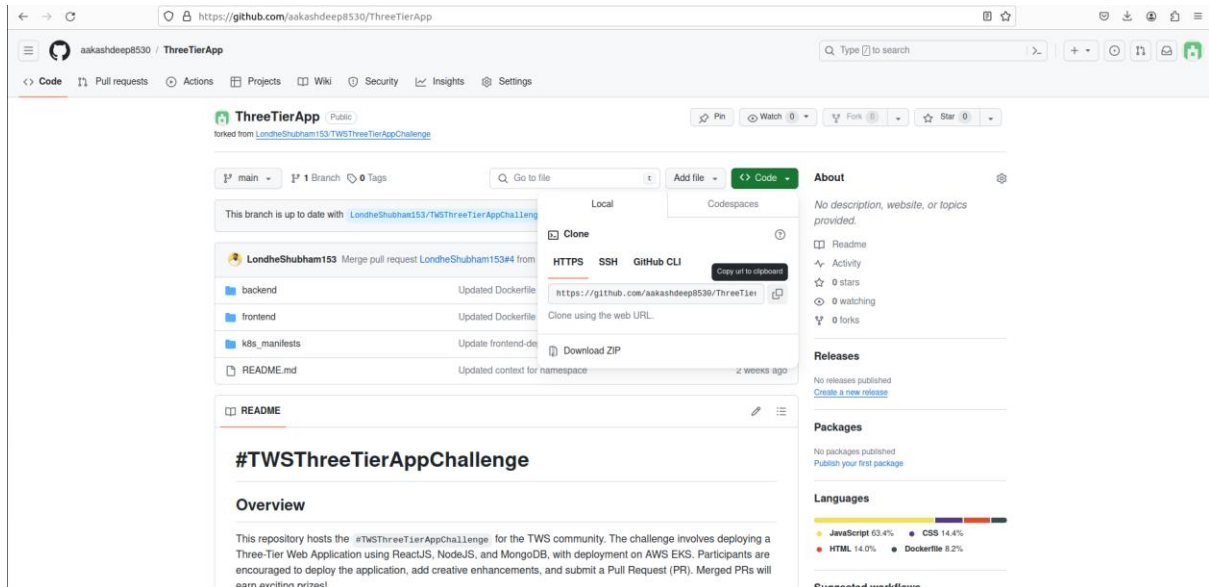
Copy Repository URL:

Open the GitHub repository containing your code.

Click on the "Code" button to reveal the repository URL.

Copy the URL provide

(it should look like <https://github.com/username/repository.git>).



Clone the Repository in terminal:

Use the git clone command followed by the copied URL:

`#git clone https://github.com/aakashdeep8530/ThreeTierApp.git`

```
ubuntu@ip-172-31-82-137:~$ git clone https://github.com/aakashdeep8530/ThreeTierApp.git
Cloning into 'ThreeTierApp'...
remote: Enumerating objects: 70, done.
remote: Counting objects: 100% (28/28), done.
remote: Compressing objects: 100% (20/20), done.
remote: Total 70 (delta 14), reused 8 (delta 8), pack-reused 42
Receiving objects: 100% (70/70), 204.97 KiB | 10.79 MiB/s, done.
Resolving deltas: 100% (17/17), done.
ubuntu@ip-172-31-82-137:~$ ls
ThreeTierApp
ubuntu@ip-172-31-82-137:~$
```

Step 1: Navigate into the ThreeTierApp directory
`cd ThreeTierApp`

Step 2: Create frontend, backend, and k8s_manifest directories
`mkdir frontend backend k8s_manifest`

Step 3: Navigate into the k8s_manifest directory
`cd k8s_manifest`

Step 4: Create a file named mongo
`touch mongo`

Frontend

STEP 3

Install Docker

#sudo apt-get update

#sudo apt install docker.io

#docker ps

```
ubuntu@ip-172-31-82-137:~/ThreeTierApp/frontend$ docker ps
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock:
Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.24/containers/json": dial unix /var/run/docker.sock: connect: permission denied
ubuntu@ip-172-31-82-137:~/ThreeTierApp/frontend$
```

#sudo chown \$USER /var/run/docker.sock

```
ubuntu@ip-172-31-82-137:~/ThreeTierApp/frontend$ docker ps
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.24/containers/json": dial unix /var/run/docker.sock: connect: permission denied
ubuntu@ip-172-31-82-137:~/ThreeTierApp/frontend$ sudo chown $USER /var/run/docker.sock
ubuntu@ip-172-31-82-137:~/ThreeTierApp/frontend$
```

Docker file

```
ubuntu@ip-172-31-82-137:~/ThreeTierApp/frontend$ vim dockerfile
ubuntu@ip-172-31-82-137:~/ThreeTierApp/frontend$ cat dockerfile
FROM node:14

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]
```

Use the official Node.js 14 image as a base image

FROM node:14

Set the working directory in the container

WORKDIR /app

Copy the package.json and package-lock.json files to the container

COPY package*.json ./

Install the application's dependencies inside the container

RUN npm install

Copy the rest of the application code to the container

COPY . .

Exposing Port to the container

EXPOSE 3000

Specify the command to run when the container starts

CMD ["npm", "start"]

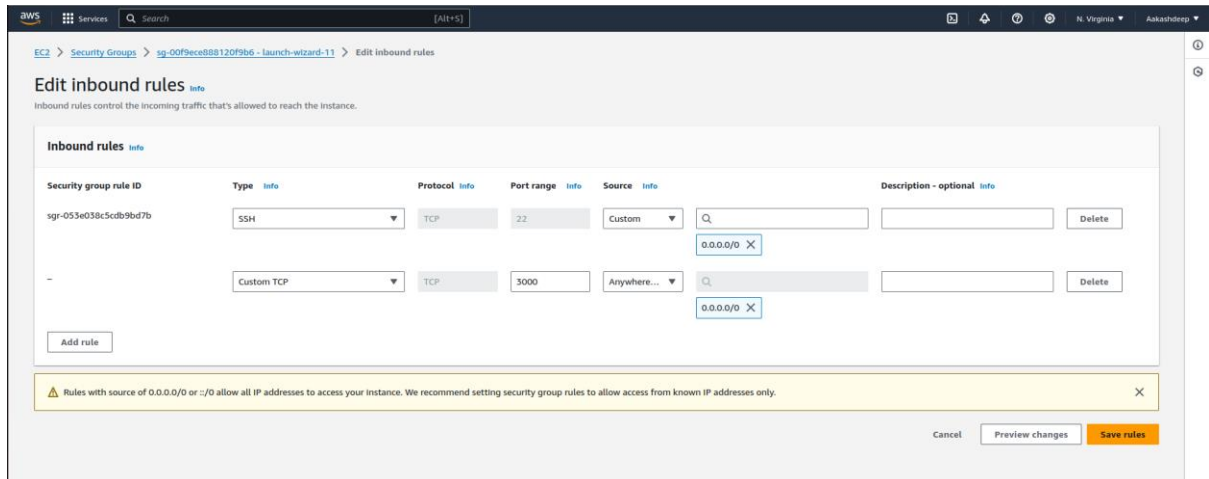
"Now, create a Docker container from the Docker image."

```
#docker build -t threetierfrontend .
```

```
#docker images
```

```
ubuntu@ip-172-31-82-137:~/ThreetierApp/frontend$ docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
threetierfrontend    latest      be1fc74beef1  About a minute ago  1.12GB
node                 14         1d12470fa662  9 months ago   912MB
```

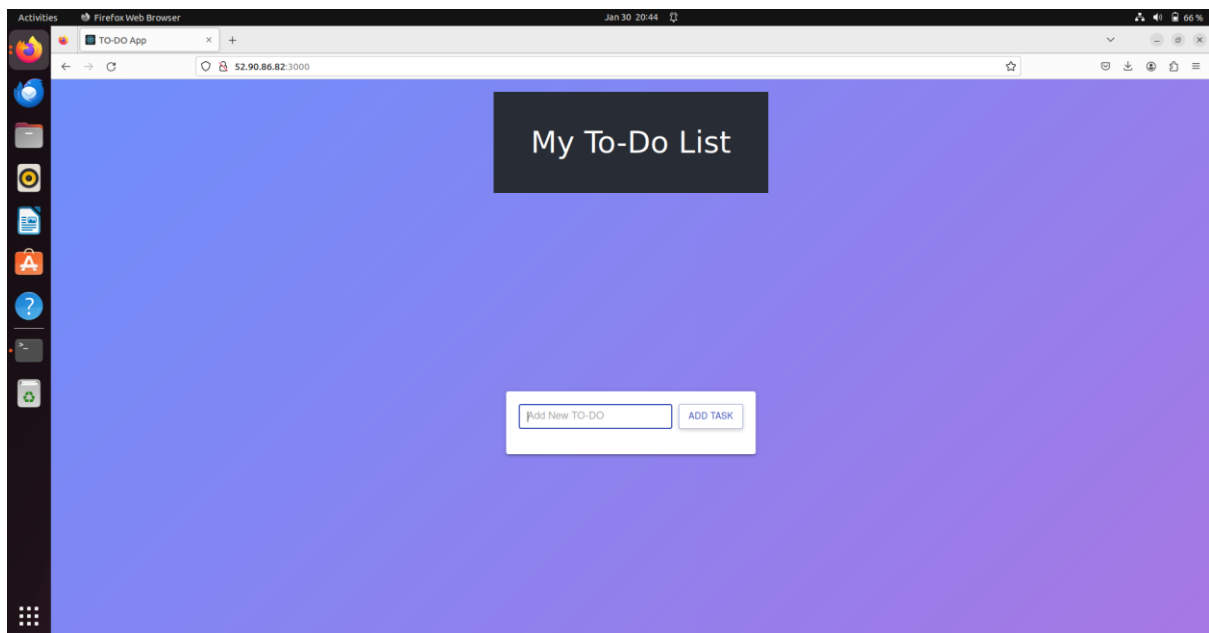
"Now, navigate to the instance's security group to enable traffic on port 3000."



```
#docker run -d -p 3000:3000 threetierfrontend:latest
```

```
ubuntu@ip-172-31-82-137:~$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS                               NAMES
dbb33aaa4ef0   threetierfrontend:latest "docker-entrypoint.s..." About an hour ago Up About an hour  0.0.0.0:3000->3000/tcp, :::3000->3000/tcp  sad_elbakyan
ubuntu@ip-172-31-82-137:~$
```

"After creation, go to the browser."



STEP 4

Install AWS CLI v2

“AWS CLI is a command-line tool for interacting with AWS services, providing developers and administrators a powerful way to manage AWS resources and automate tasks. It supports cross-platform use, structured output formats, and enables scripting for resource management across various AWS services.”

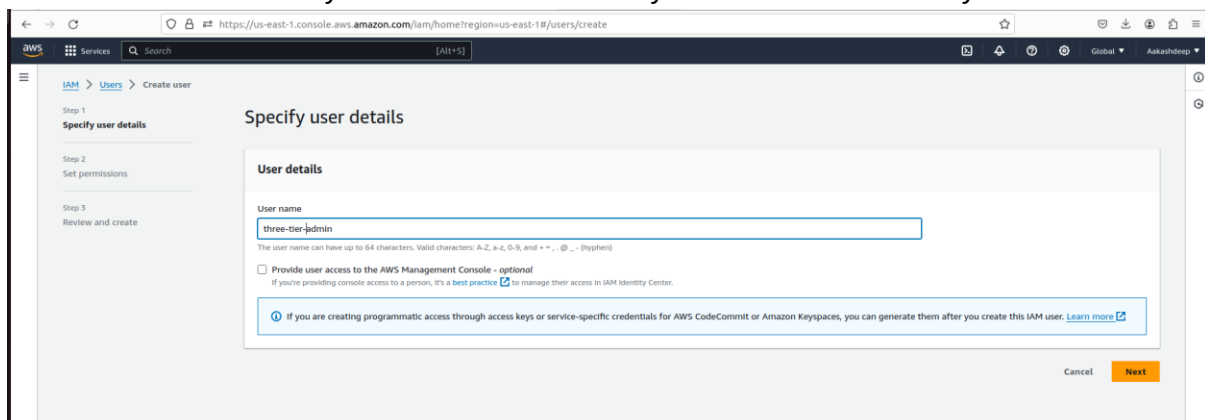
```
#curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
#sudo apt install unzip
#unzip awscliv2.zip
#sudo ./aws/install -i /usr/local/aws-cli -b /usr/local/bin --update
#aws configure
```

STEP 5

IAM Configuration

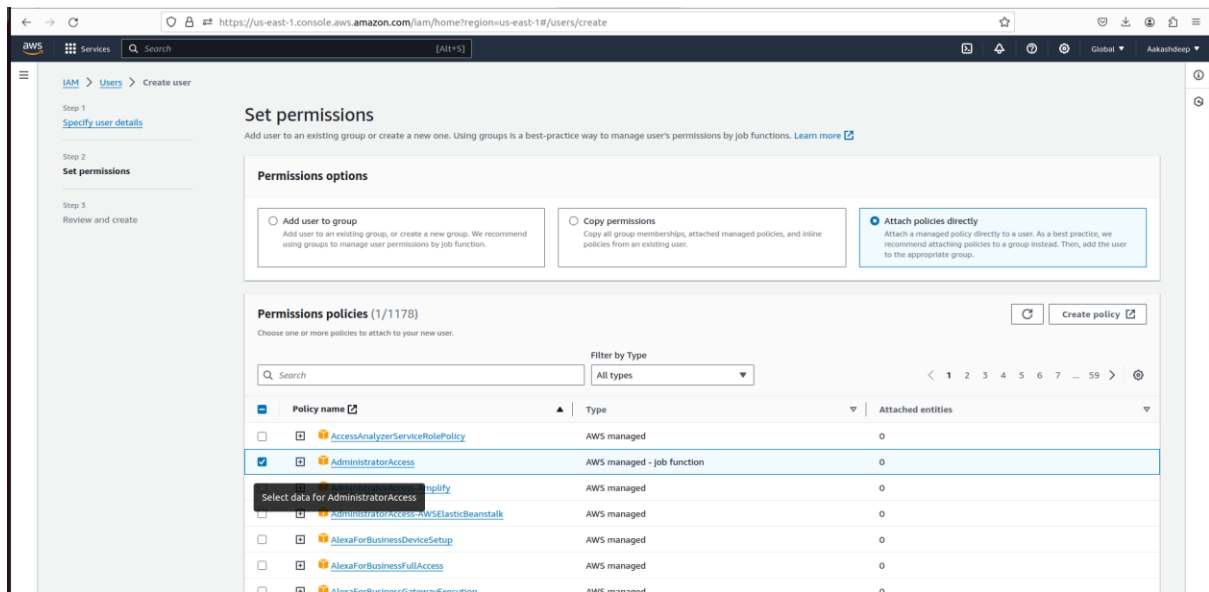
AWS Identity and Access Management (IAM) is used to securely manage access to AWS services and resources. It allows you to create and control AWS users and groups, define fine-grained permissions, and enhance security by enforcing the principle of least privilege within your AWS environment.

- Create a user **three-tier-admin** with **AdministratorAccess**.
- Generate Security Credentials: Access Key and Secret Access Key

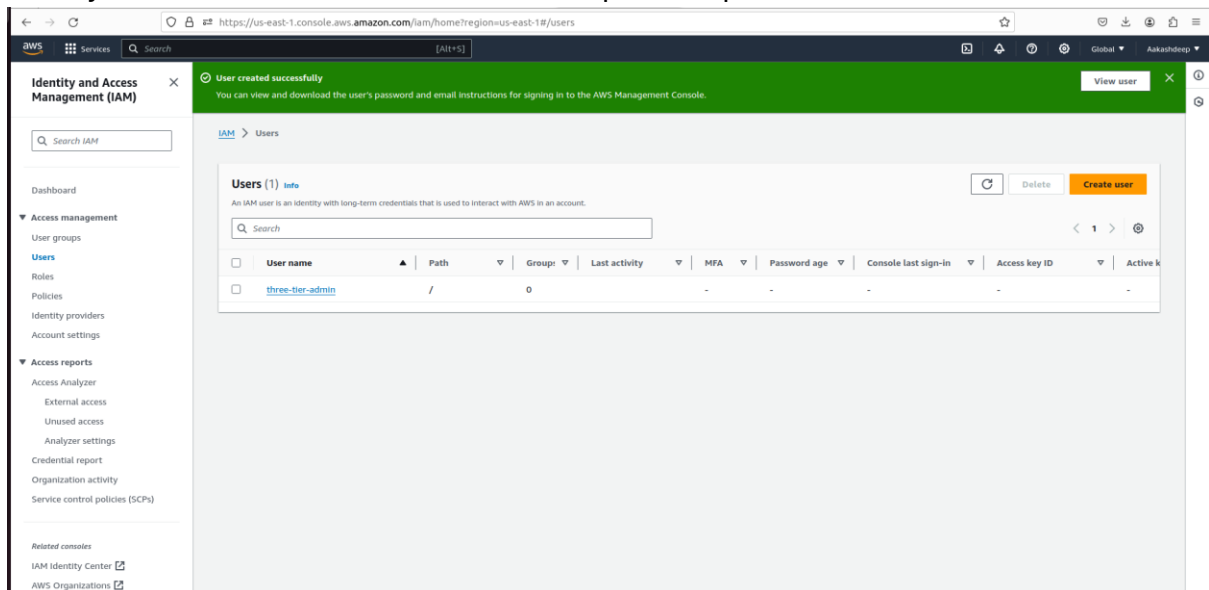


The screenshot shows the AWS IAM console interface for creating a new user. The browser address bar indicates the URL: `https://us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/users/create`. The left sidebar shows the navigation menu with 'IAM' selected, and the 'Users' section expanded. The main content area is titled 'Specify user details' and contains a 'User details' section. In this section, the 'User name' field is populated with 'three-tier-admin'. Below this field, there is a checkbox labeled 'Provide user access to the AWS Management Console - optional', which is currently unchecked. A note below the checkbox states: 'If you're providing console access to a person, it's a best practice to manage their access in IAM Identity Center.' At the bottom of the 'User details' section, there is a blue box with an information icon and text: 'If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user. [Learn more](#)'. At the bottom right of the page, there are 'Cancel' and 'Next' buttons.

"After specifying the user details, proceed to set permissions by navigating to 'Attach policies' and select 'AdministratorAccess' to grant the user full administrative privileges."



"Finally, select the 'Create user' button to complete the process."



"After creating the user, navigate to 'Security credentials,' click on 'Create access key' in the 'Access keys' section, and then select 'CLI' for programmatic access. After selecting 'CLI' in the 'Create access key' step, proceed to create the access key by clicking the 'Create access key' button."

three-tier-admin

Summary

ARN arn:aws:iam::695171514664:user/three-tier-admin	Console access Disabled	Access key 1 Create access key
Created January 30, 2024, 21:24 (UTC+05:30)	Last console sign-in -	

Permissions Groups Tags **Security credentials** Access Advisor

Console sign-in [Enable console access](#)

Console sign-in link
<https://695171514664.signin.aws.amazon.com/console>

Console password
Not enabled

Multi-factor authentication (MFA) (0) [Remove](#) [Resync](#) [Assign MFA device](#)

Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 8 MFA devices assigned. [Learn more](#)

Device type	Identifier	Certifications	Created on
No MFA devices. Assign an MFA device to improve the security of your AWS environment.			

[Assign MFA device](#)

Access key best practices & alternatives

Avoid using long-term credentials like access keys to improve your security. Consider the following use cases and alternatives.

Use case

- ☒ **Command Line Interface (CLI)**
You plan to use this access key to enable the AWS CLI to access your AWS account.
- ☐ **Local code**
You plan to use this access key to enable application code in a local development environment to access your AWS account.
- ☐ **Application running on an AWS compute service**
You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.
- ☐ **Third-party service**
You plan to use this access key to enable access for a third-party application or service that monitors or manages your AWS resources.
- ☐ **Application running outside AWS**
You plan to use this access key to authenticate workloads running in your data center or other infrastructure outside of AWS that needs to access your AWS resources.
- ☐ **Other**
Your use case is not listed here.

Alternatives recommended

- Use [AWS CloudShell](#), a browser-based CLI, to run commands. [Learn more](#)
- Use the [AWS CLI V2](#) and enable authentication through a user in IAM Identity Center. [Learn more](#)

Access key created
This is the only time that the secret access key can be viewed or downloaded. You cannot recover it later. However, you can create a new access key any time.

Retrieve access keys

Access key
If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key	Secret access key
AKIAZDWSTQEUBWLSJLQK	***** Show

Access key best practices

- Never store your access key in plain text, in a code repository, or in code.
- Disable or delete access key when no longer needed.
- Enable least-privilege permissions.
- Rotate access keys regularly.

For more details about managing access keys, see the [best practices for managing AWS access keys](#).

[Download .csv file](#) [Done](#)

"After obtaining the keys, open the terminal and run `aws configure`. Provide the access key and secret key when prompted."

```
ubuntu@ip-172-31-82-137:~$ aws configure
AWS Access Key ID [*****JLGK]:
AWS Secret Access Key [*****QVjf]:
Default region name [None]:
Default output format [None]:
ubuntu@ip-172-31-82-137:~$
```

STEP 6

Amazon Elastic Container Registry

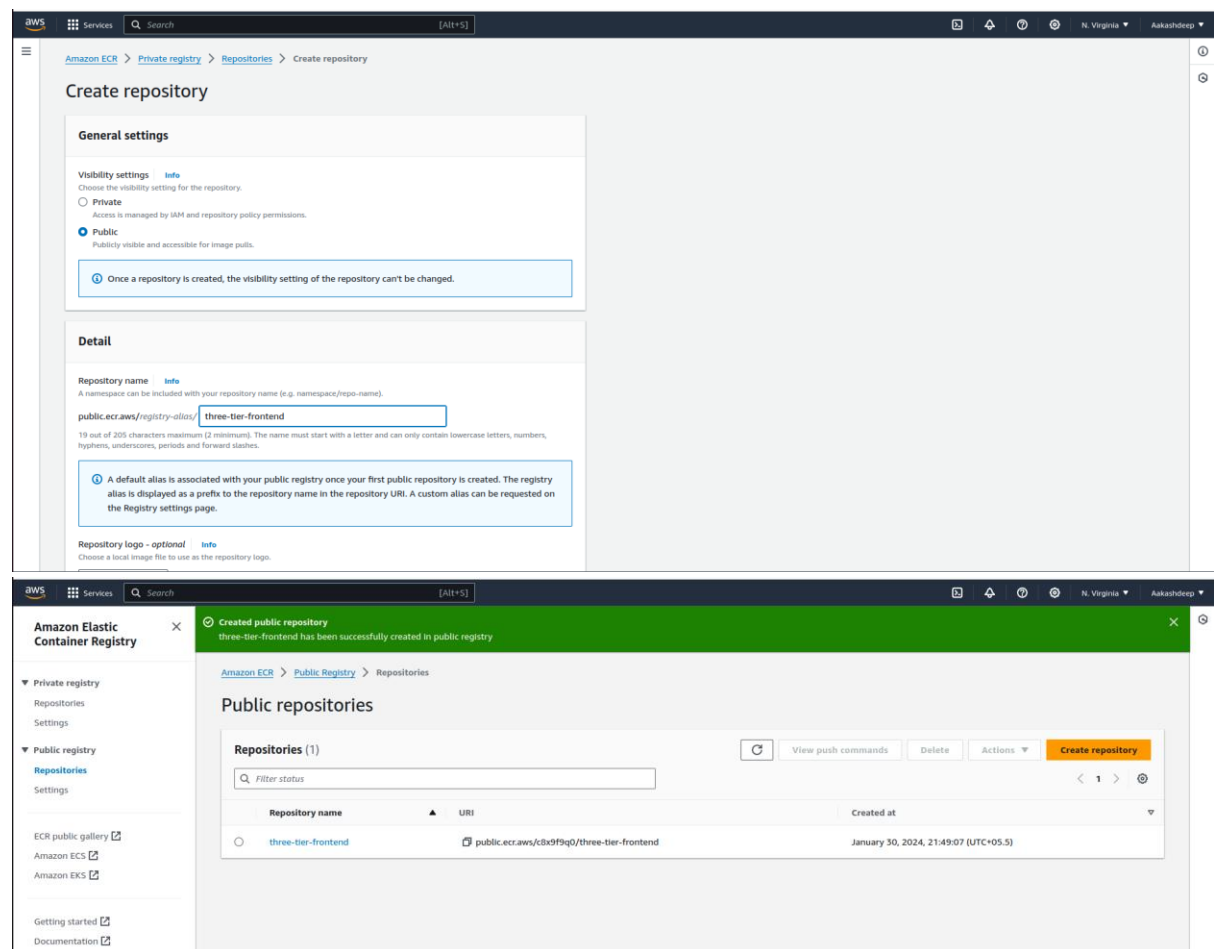
(ECR) is a fully managed container registry service provided by AWS. It allows users to store, manage, and deploy Docker container images. ECR integrates with other AWS services, such as Amazon ECS (Elastic Container Service), making it easier to build, store, and deploy containerized applications in the AWS ecosystem.

Create a Repository:

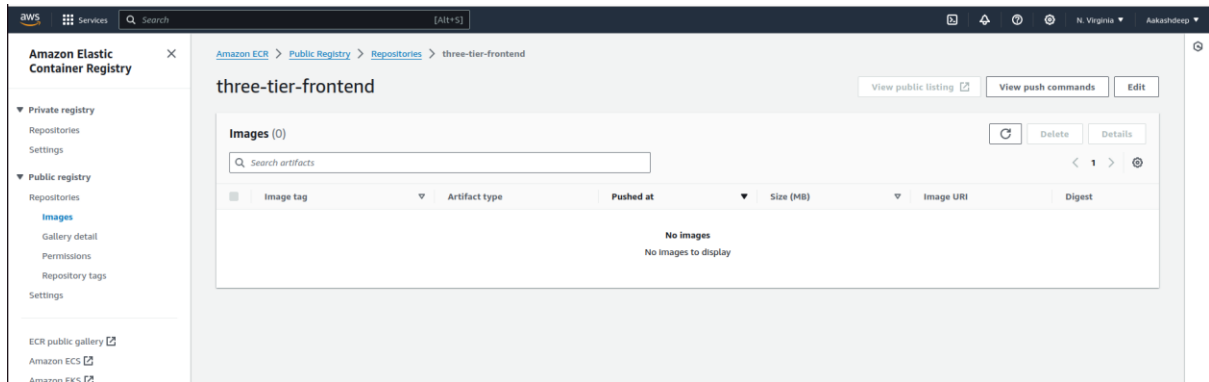
In the ECR dashboard, click on the **"Create repository"** button.

Enter a repository name (e.g., my-docker-repo).

Optionally, add a tag for the repository.

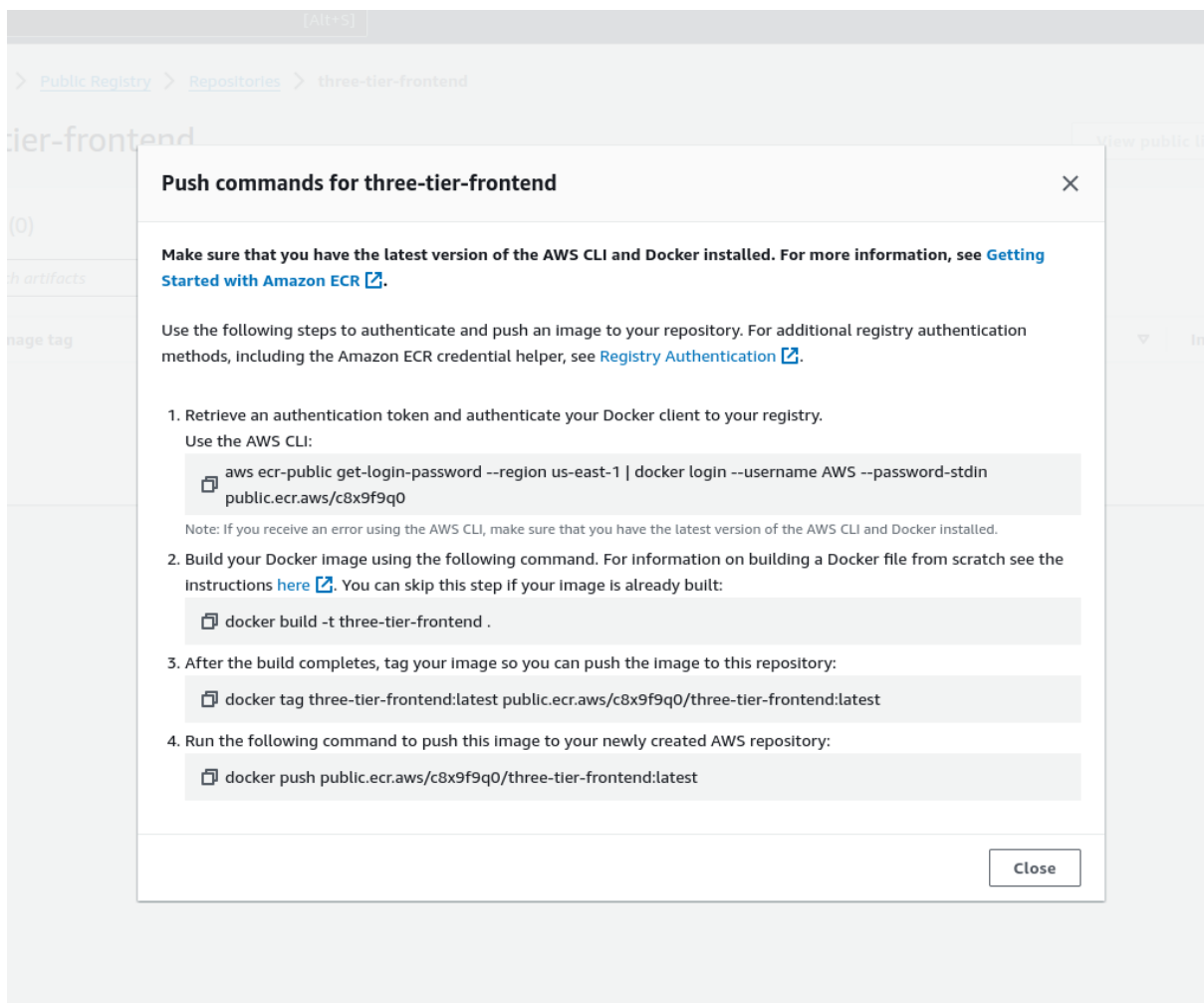


After creating the ECR repository, open the created repository.



The repository is currently empty as no images have been pushed to it.

To get the push command, click the "View push commands" button in the upper-right corner and follow the instructions provided.



```

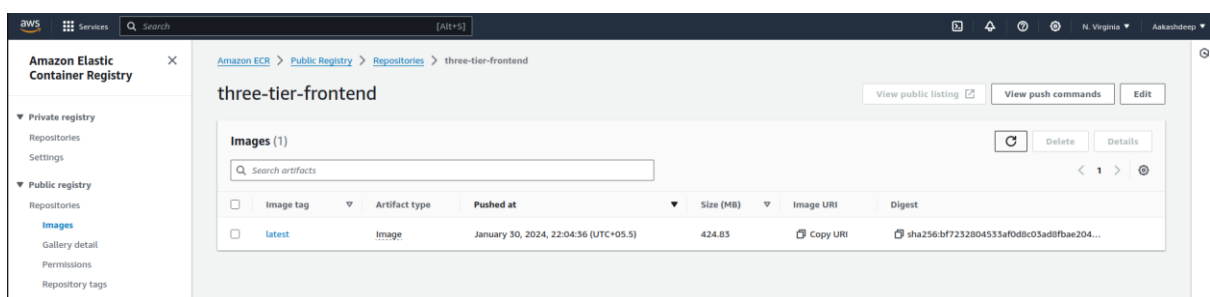
ubuntu@ip-172-31-82-137:~$ aws ecr-public get-login-password --region us-east-1 | docker login --username AWS --password-stdin public.ecr.aws/c8x9f9q0
WARNING! Your password will be stored unencrypted in /home/ubuntu/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
ubuntu@ip-172-31-82-137:~$ cd ThreeTierApp/
ubuntu@ip-172-31-82-137:~/ThreeTierApp$ cd frontend/
ubuntu@ip-172-31-82-137:~/ThreeTierApp/frontend$ ls
dockerfile  package-lock.json  package.json  public  src
ubuntu@ip-172-31-82-137:~/ThreeTierApp/frontend$ docker build -t three-tier-frontend .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 739.3kB
Step 1/7 : FROM node:14
--> 1d12470fa662
Step 2/7 : WORKDIR /app
--> Using cache
--> 6d73ec1d7d99
Step 3/7 : COPY package*.json ./
--> Using cache
--> c1399fa20d2e
Step 4/7 : RUN npm install
--> Using cache
--> 47edc114f38a
Step 5/7 : COPY . .
--> Using cache
--> 31476df2b7c4
Step 6/7 : EXPOSE 3000
--> Using cache
--> 39e2711b8a39
Step 7/7 : CMD ["npm", "start"]
--> Using cache
--> be1fc74beef1
Successfully built be1fc74beef1
Successfully tagged three-tier-frontend:latest
ubuntu@ip-172-31-82-137:~/ThreeTierApp/frontend$ docker tag three-tier-frontend:latest public.ecr.aws/c8x9f9q0/three-tier-frontend:latest
ubuntu@ip-172-31-82-137:~/ThreeTierApp/frontend$ docker push public.ecr.aws/c8x9f9q0/three-tier-frontend:latest
The push refers to repository [public.ecr.aws/c8x9f9q0/three-tier-frontend]
04bc9dcbb539: Pushed
7e4aac5bf17f: Pushed
2e3ac8b0b86d: Pushed
2a1a5f7dd9fa: Pushed
0d5f5a015e5d: Pushed
3c777d951de2: Pushed
f8a91dd5fc84: Pushed
cb81227abde5: Pushed
e01a454893a9: Pushed
c45660adde37: Pushed
fe0fb3ab4a0f: Pushed
f1186e5061f2: Pushed
b2dba7477754: Pushed
latest: digest: sha256:bf7232804533af0d8c03ad8fbae2042da107f784dde79b853056c5878f36647a size: 3053
ubuntu@ip-172-31-82-137:~/ThreeTierApp/frontend$

```

After successfully pushing the image to ECR, you should see the image listed in the ECR repository.



Backend

STEP 7

Docker file

```
ubuntu@ip-172-31-82-137:~/ThreeTierApp/backend$ vim dockerfile
ubuntu@ip-172-31-82-137:~/ThreeTierApp/backend$ cat dockerfile
FROM node:14

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 8080

CMD [ "node", "index.js" ]
ubuntu@ip-172-31-82-137:~/ThreeTierApp/backend$
```

Use the official Node.js 14 image as a base image

FROM node:14

Set the working directory in the container

WORKDIR /app

Copy the package.json and package-lock.json files to the container

COPY package*.json ./

Install the application's dependencies inside the container

RUN npm install

Copy the rest of the application code to the container

COPY . .

Expose Port to the container

EXPOSE 8080

Specify the command to run when the container starts

CMD ["node", "index.js"]

docker build -t three-tier-backend .

#docker run -d -p 8080:8080 three-tier-backend:latest

```
ubuntu@ip-172-31-82-137:~/ThreeTierApp/backend$ docker run -d -p 8080:8080 three-tier-backend:latest
5a21f3dfbc46d77c90e4a2a7d74c194ef2f080be484f5ccfb6fa4896573391dd
ubuntu@ip-172-31-82-137:~/ThreeTierApp/backend$
```

To push the three-tier-backend Docker image to Amazon ECR, follow these steps:

Back

Push commands for three-tier-backend

Make sure that you have the latest version of the AWS CLI and Docker installed. For more information, see [Getting Started with Amazon ECR](#).

Use the following steps to authenticate and push an image to your repository. For additional registry authentication methods, including the Amazon ECR credential helper, see [Registry Authentication](#).

1. Retrieve an authentication token and authenticate your Docker client to your registry.

Use the AWS CLI:

aws ecr-public get-login-password --region us-east-1 | docker login --username AWS --password-stdin public.ecr.aws/c8x9f9q0

Note: If you receive an error using the AWS CLI, make sure that you have the latest version of the AWS CLI and Docker installed.

2. Build your Docker image using the following command. For information on building a Docker file from scratch see the instructions [here](#). You can skip this step if your image is already built:

docker build -t three-tier-backend .

3. After the build completes, tag your image so you can push the image to this repository:

docker tag three-tier-backend:latest public.ecr.aws/c8x9f9q0/three-tier-backend:latest

4. Run the following command to push this image to your newly created AWS repository:

docker push public.ecr.aws/c8x9f9q0/three-tier-backend:latest

Close

Amazon Elastic Container Registry

Private registry

Public registry

three-tier-backend

View public listing

View push commands

Edit

Images (1)

Search artifacts

Image tag

Artifact type

Pushed at

Size (MB)

Image URI

Digest

latest

Image

January 30, 2024, 23:01:26 (UTC+05.5)

353.94

Copy URI

sha256:71b653b7a6d4e09371a48d3a804ca...

The **docker logs** command is used to view the logs generated by a running Docker container.

#docker logs <container id>

```
ubuntu@ip-172-31-82-137:~/ThreeTierApp/backend$ docker logs 5a21f3dfbc46
Listening on port 8080...
Could not connect to database. MongooseError: The `uri` parameter to `openUri()` must be a string, got "undefined". Make sure the
first parameter to `mongoose.connect()` or `mongoose.createConnection()` is a string.
    at NativeConnection.Connection.openUri (/app/node_modules/mongoose/lib/connection.js:694:11)
    at /app/node_modules/mongoose/lib/index.js:351:10
    at /app/node_modules/mongoose/lib/helpers/promiseOrCallback.js:32:5
    at new Promise (<anonymous>)
    at promiseOrCallback (/app/node_modules/mongoose/lib/helpers/promiseOrCallback.js:31:10)
    at Mongoose._promiseOrCallback (/app/node_modules/mongoose/lib/index.js:1149:10)
    at Mongoose.connect (/app/node_modules/mongoose/lib/index.js:350:20)
    at module.exports (/app/db.js:17:24)
    at Object.<anonymous> (/app/index.js:7:1)
    at Module._compile (internal/modules/cjs/loader.js:1114:14)
    at Object.Module._extensions..js (internal/modules/cjs/loader.js:1143:10)
    at Module.load (internal/modules/cjs/loader.js:979:32)
    at Function.Module._load (internal/modules/cjs/loader.js:819:12)
    at Function.executeUserEntryPoint [as runMain] (internal/modules/run_main.js:75:12)
    at internal/main/run_main_module.js:17:47
ubuntu@ip-172-31-82-137:~/ThreeTierApp/backend$
```

“The error indicates that there's an issue with connecting to the MongoDB database, which can lead to errors in database interactions within the application.”

STEP 8

14

Install kubectl

```
#curl -o kubectl https://amazon-eks.s3.us-west-2.amazonaws.com/1.19.6/2021-01-05/bin/linux/amd64/kubectl
#chmod +x ./kubectl
#sudo mv ./kubectl /usr/local/bin
#kubectl version --short --client
```

STEP 9

Install eksctl

```
#curl --silent --location
"https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_$(uname -
s)_amd64.tar.gz" | tar xz -C /tmp
#sudo mv /tmp/eksctl /usr/local/bin
#eksctl version
```

STEP 10

Setup EKS Cluster in the region us-west-1 with t2.medium with min nod 2 and max with 2

```
#eksctl create cluster --name three-tier-cluster --region us-east-1 --node-type t2.medium --
nodes-min 2 --nodes-max 2
```

```
ubuntu@ip-172-31-82-137:~$ eksctl create cluster --name three-tier-cluster --region us-east-1 --node-type t2.medium --nodes-min 2 --nodes-max 2
2024-01-30 18:17:07 [i] eksctl version 0.169.0
2024-01-30 18:17:07 [i] using region us-east-1
2024-01-30 18:17:07 [i] setting availability zones to [us-east-1b us-east-1d]
2024-01-30 18:17:07 [i] subnets for us-east-1b - public:192.168.0.0/19 private:192.168.64.0/19
2024-01-30 18:17:07 [i] subnets for us-east-1d - public:192.168.32.0/19 private:192.168.96.0/19
2024-01-30 18:17:07 [i] nodegroup "ng-88778ab2" will use "" [AmazonLinux2/1.27]
2024-01-30 18:17:07 [i] using Kubernetes version 1.27
2024-01-30 18:17:07 [i] creating EKS cluster "three-tier-cluster" in "us-east-1" region with managed nodes
2024-01-30 18:17:07 [i] will create 2 separate CloudFormation stacks for cluster itself and the initial managed nodegroup
2024-01-30 18:17:07 [i] if you encounter any issues, check CloudFormation console or try 'eksctl utils describe-stacks --region=us-east-1 --cluster=three-tier-cluster'
2024-01-30 18:17:07 [i] Kubernetes API endpoint access will use default of {publicAccess=true, privateAccess=false} for cluster "three-tier-cluster" in "us-east-1"
2024-01-30 18:17:07 [i] CloudWatch logging will not be enabled for cluster "three-tier-cluster" in "us-east-1"
2024-01-30 18:17:07 [i] you can enable it with 'eksctl utils update-cluster-logging --enable-types={SPECIFY-YOUR-LOG-TYPES-HERE (e.g. all)} --region=us-east-1 --cluster=three-tier-clus
2024-01-30 18:17:07 [i]
2 sequential tasks: { create cluster control plane "three-tier-cluster",
                      wait for control plane to become ready,
                      create managed nodegroup "ng-88778ab2",
```

"This information can be viewed in the AWS CloudFormation console."

CloudFormation > **Stacks** > eksctl-three-tier-cluster-cluster

Stacks (1)

Filter status: Active View nested

Stacks

- eksctl-three-tier-cluster-cluster
 - 2024-01-30 23:47:07 UTC+0530
 - CREATE_IN_PROGRESS

eksctl-three-tier-cluster-cluster

Stack info | Events | **Resources** | Outputs | Parameters | Template | Change sets | Git sync - new

Resources (26)

Search resources

Logical ID	Physical ID	Type	Status	Module
ClusterSharedNodeSecurityGroup	sg-08c8a7648357234b3	AWS::EC2::SecurityGroup	CREATE_COMPLETE	-
ControlPlane	three-tier-cluster	AWS::EKS::Cluster	CREATE_IN_PROGRESS	-
ControlPlaneSecurityGroup	sg-02b3a3ac2308d7d4b	AWS::EC2::SecurityGroup	CREATE_COMPLETE	-
IngressInterNodeGroupSG	IngressInterNodeGroupSG	AWS::EC2::SecurityGroup	CREATE_COMPLETE	-
InternetGateway	igw-09ee5e2491e36d2ab	AWS::EC2::InternetGateway	CREATE_COMPLETE	-
NATGateway	nat-0a0278bd1b91c014d	AWS::EC2::NatGateway	CREATE_COMPLETE	-
NATIP	52.1.138.202	AWS::EC2::EIP	CREATE_COMPLETE	-
NATPrivateSubnetRouteUSEAST1B	rtb-028173ef00ed72200j0.0.0/0	AWS::EC2::Route	CREATE_COMPLETE	-
NATPrivateSubnetRouteUSEAST1D	rtb-070a51791ba3c720dj0.0.0/0	AWS::EC2::Route	CREATE_COMPLETE	-
PolicyCloudWatchMetrics	eksctl-Polic-j9q5zreQbAHV	AWS::IAM::Policy	CREATE_COMPLETE	-

Amazon Elastic Kubernetes Service(EKS), is a managed Kubernetes service provided by Amazon Web Services (AWS). Kubernetes is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications.

Amazon Elastic Kubernetes Service

Clusters New

Amazon EKS Anywhere
Enterprise Subscriptions New

Related services
Amazon ECR
AWS Batch

Documentation
Submit feedback

Cluster version update in progress
The Kubernetes version for three-tier-cluster is now being updated. This process may take several minutes. When the version update completes, update all node groups and worker nodes to match the new cluster version.

View documentation

EKS > Clusters

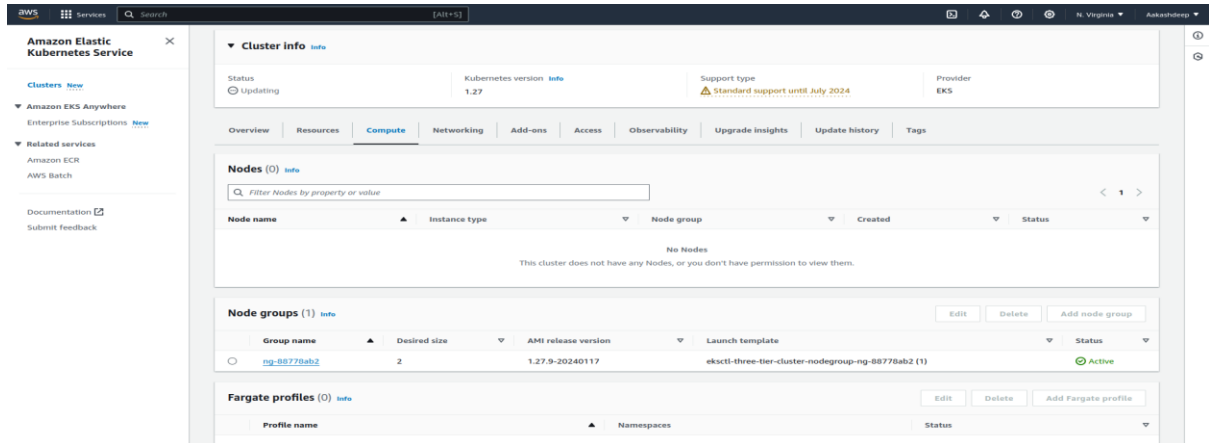
New Kubernetes versions are available for 1 cluster.

Clusters (1) info

Filter clusters

Cluster name	Status	Kubernetes version	Support type	Provider
three-tier-cluster	Active	1.27 Update now	Standard support until July 2024	EKS

"Here, two nodes are created in the Kubernetes cluster."



```
#aws eks update-kubeconfig --region us-east-1 --name three-tier-cluster
```

```
#kubectl get nodes
```

```
ubuntu@ip-172-31-82-137:~$ aws eks update-kubeconfig --region us-east-1 --name three-tier-cluster
Added new context arn:aws:eks:us-east-1:695171514664:cluster/three-tier-cluster to /home/ubuntu/.kube/c
ubuntu@ip-172-31-82-137:~$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-192-168-28-185.ec2.internal      Ready    <none>   11m   v1.27.9-eks-5e0fdde
ip-192-168-52-171.ec2.internal      Ready    <none>   11m   v1.27.9-eks-5e0fdde
```

Database

STEP 11

Create namespace and Run Manifests files

In Kubernetes, a **namespace** is a virtual environment that organizes resources, separating teams or projects within the same cluster. It enhances resource management, isolation, and overall organization.

```
#kubectl create namespace workshop
```

```
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests/mongo$ kubectl create namespace workshop
namespace/workshop created
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests/mongo$
```

deploy.yaml

```

ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests/mongo$ cat deploy.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  namespace: workshop
  name: mongodb
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongodb
  template:
    metadata:
      labels:
        app: mongodb
    spec:
      containers:
        - name: mongodb
          image: mongo:4.4.6
          command:
            - "numactl"
            - "--interleave=all"
            - "mongod"
            - "--wiredTigerCacheSizeGB"
            - "0.1"
            - "--bind_ip"
            - "0.0.0.0"
          ports:
            - containerPort: 27017
          resources:
            requests:
              memory: "512Mi"
              cpu: "250m"
            limits:
              memory: "1Gi"
              cpu: "500m"
          env:
            - name: MONGO_INITDB_ROOT_USERNAME
              valueFrom:
                secretKeyRef:
                  name: mongo-sec
                  key: username
            - name: MONGO_INITDB_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mongo-sec
                  key: password

```

secrets.yaml

```

ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests/mongo$ cat secrets.yaml
apiVersion: v1
kind: Secret
metadata:
  namespace: workshop
  name: mongo-sec
type: Opaque
data:
  password: YWRtaW5AMTIzCg== #admin@123
  username: YWRtaW4= #admin

```

service.yaml

```
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests/mongo$ cat service.yaml
apiVersion: v1
kind: Service
metadata:
  namespace: workshop
  name: mongodb-svc
spec:
  selector:
    app: mongodb
  ports:
  - name: mongodb-svc
    protocol: TCP
    port: 27017
    targetPort: 27017ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests/mongo$
```

```
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests/mongo$ ls
deploy.yaml  secrets.yaml  service.yaml
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests/mongo$ kubectl apply -f deploy.yaml
deployment.apps/mongodb created
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests/mongo$ kubectl apply -f secrets.yaml
secret/mongo-sec created
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests/mongo$ kubectl apply -f service.yaml
service/mongodb-svc created
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests/mongo$ kubectl get svc -n workshop
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
mongodb-svc	ClusterIP	10.100.229.30	<none>	27017/TCP	25s

backend-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: workshop
  labels:
    role: api
    env: demo
spec:
  replicas: 1
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 25%
  selector:
    matchLabels:
      role: api
  template:
    metadata:
      labels:
        role: api
    spec:
      containers:
      - name: api
        image: public.ecr.aws/c8x9f9q0/three-tier-backend:latest
        imagePullPolicy: Always
        env:
        - name: MONGO_CONN_STR
          value: mongodb://mongodb-svc:27017/todo?directConnection=true
        - name: MONGO_USERNAME
```

```

valueFrom:
secretKeyRef:
name: mongo-sec
key: username
- name: MONGO_PASSWORD
valueFrom:
secretKeyRef:
name: mongo-sec
key: password
ports:
- containerPort: 8080
livenessProbe:
httpGet:
path: /ok
port: 8080
initialDelaySeconds: 2
periodSeconds: 5
readinessProbe:
httpGet:
path: /ok
port: 8080
initialDelaySeconds: 5
periodSeconds: 5
successThreshold: 1

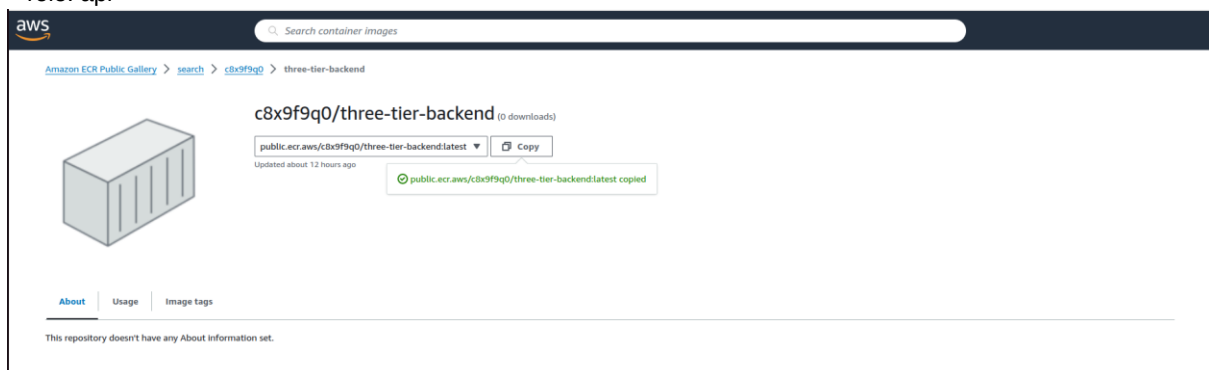
```

backend-service.yaml

```

---
apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: workshop
spec:
  ports:
  - port: 8080
    protocol: TCP
  type: ClusterIP
selector:
  role: api

```



```

    role: api
  spec:
    containers:
    - name: api
      image: public.ecr.aws/c8x9f9q0/three-tier-backend:latest
      imagePullPolicy: Always
      env:
        - name: MONGO_CONN_STR
          value: mongodb://mongodb-svc:27017/todo?directConnection=true
        - name: MONGO_USERNAME
          valueFrom:
            secretKeyRef:
              name: mongo-sec
              key: username

```

```

ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests$ kubectl apply -f backend-deployment.yaml
deployment.apps/api created
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests$ kubectl apply -f backend-service.yaml
service/api created
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests$ kubectl get pods -n workshop
NAME                                READY   STATUS    RESTARTS   AGE
api-7b47b5f8c6-hv6tt               1/1     Running   0           53s
mongodb-7f58c5f5d9-7cstv           1/1     Running   0           146m

```

Now connect to database

```

ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests$ kubectl logs api-7b47b5f8c6-hv6tt -n workshop
Listening on port 8080...
Connected to database.
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests$

```

frontend-deployment.yaml

```

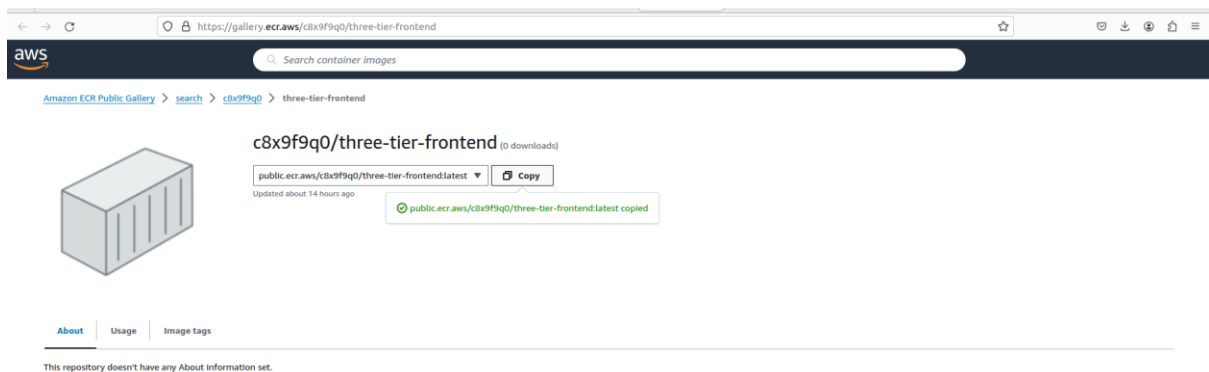
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: workshop
  labels:
    role: frontend
    env: demo
spec:
  replicas: 1
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 25%
  selector:
    matchLabels:
      role: frontend
  template:
    metadata:
      labels:
        role: frontend
    spec:
      containers:

```

```
- name: frontend
image: public.ecr.aws/c8x9f9q0/three-tier-frontend:latest
imagePullPolicy: Always
env:
- name: REACT_APP_BACKEND_URL
value: "http://app.trainwithshubham.com/api/tasks"
ports:
- containerPort: 3000
```

frontend-service.yaml

```
---
apiVersion: v1
kind: Service
metadata:
  name: frontend
  namespace: workshop
spec:
  ports:
  - port: 3000
    protocol: TCP
  type: ClusterIP
selector:
  role: frontend
```



```
role: frontend
spec:
  containers:
  - name: frontend
    image: public.ecr.aws/c8x9f9q0/three-tier-frontend:latest
    imagePullPolicy: Always
    env:
      - name: REACT_APP_BACKEND_URL
        value: "http://app.trainwithshubham.com/api/tasks"
    ports:
      - containerPort: 3000
```

```

service/frontend created
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests$ kubectl get pods -n workshop
NAME                                READY   STATUS    RESTARTS   AGE
api-7b47b5f8c6-hv6tt               1/1     Running   0           17m
frontend-7bbb9db77c-lv4cl           1/1     Running   0           44s
mongodb-7f58c5f5d9-7cstv            1/1     Running   0           162m
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests$

```

STEP 12

Install AWS Load Balancer

```
#curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.5.4/docs/install/iam\_policy.json
```

```
#aws iam create-policy --policy-name AWSLoadBalancerControllerIAMPolicy --policy-document file://iam_policy.json
```

```
#eksctl utils associate-iam-oidc-provider --region=us-east-1 --cluster=three-tier-cluster --approve
```

```
#eksctl create iamserviceaccount --cluster=three-tier-cluster --namespace=kube-system --name=aws-load-balancer-controller --role-name AmazonEKSLoadBalancerControllerRole --attach-policy-arn=arn:aws:iam::695171514664:policy/AWSLoadBalancerControllerIAMPolicy --approve --region=us-east-1
```

STEP 13

Install Helm and Deploy AWS Load Balancer Controller

```
#sudo snap install helm --classic
#helm repo add eks https://aws.github.io/eks-charts
#helm repo update eks
#helm install aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system --set clusterName=three-tier-cluster --set serviceAccount.create=false --set serviceAccount.name=aws-load-balancer-controller
#kubectl get deployment -n kube-system aws-load-balancer-controller
```

```

ubuntu@ip-172-31-82-137:~$ sudo snap install helm --classic
helm 3.14.0 from Snapcrafters installed
ubuntu@ip-172-31-82-137:~$ helm repo add eks https://aws.github.io/eks-charts
"eks" has been added to your repositories
ubuntu@ip-172-31-82-137:~$ helm repo update eks
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "eks" chart repository
Update Complete. 🎉Happy Helming!🎉
ubuntu@ip-172-31-82-137:~$ helm install aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system --set clusterName=three-tier-cluster --set serviceAccount.create=false --set serviceAccount.name=aws-load-balancer-controller
NAME: aws-load-balancer-controller
LAST DEPLOYED: Wed Jan 31 07:02:21 2024
NAMESPACE: kube-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
AWS Load Balancer controller installed!
ubuntu@ip-172-31-82-137:~$ kubectl get deployment -n kube-system aws-load-balancer-controller
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
aws-load-balancer-controller        2/2     2             2           6m5s
ubuntu@ip-172-31-82-137:~$

```

full_stack_lb.yaml

```
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mainlb
  namespace: workshop
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}]'
spec:
  ingressClassName: alb
  rules:
    - host: app.trainwithshubham.com
      http:
        paths:
          - path: /api
            pathType: Prefix
          backend:
            service:
              name: api
              port:
                number: 8080
    - path: /
      pathType: Prefix
      backend:
        service:
          name: frontend
          port:
            number: 3000
```

#kubectl apply -f full_stack_lb.yaml

```
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests$ kubectl apply -f full_stack_lb.yaml
ingress.networking.k8s.io/mainlb created
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests$
```

```
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests$ kubectl get ing -n workshop
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
mainlb	alb	app.trainwithshubham.com	k8s-workshop-mainlb-7f430e3450-885677032.us-east-1.elb.amazonaws.com	80	8h

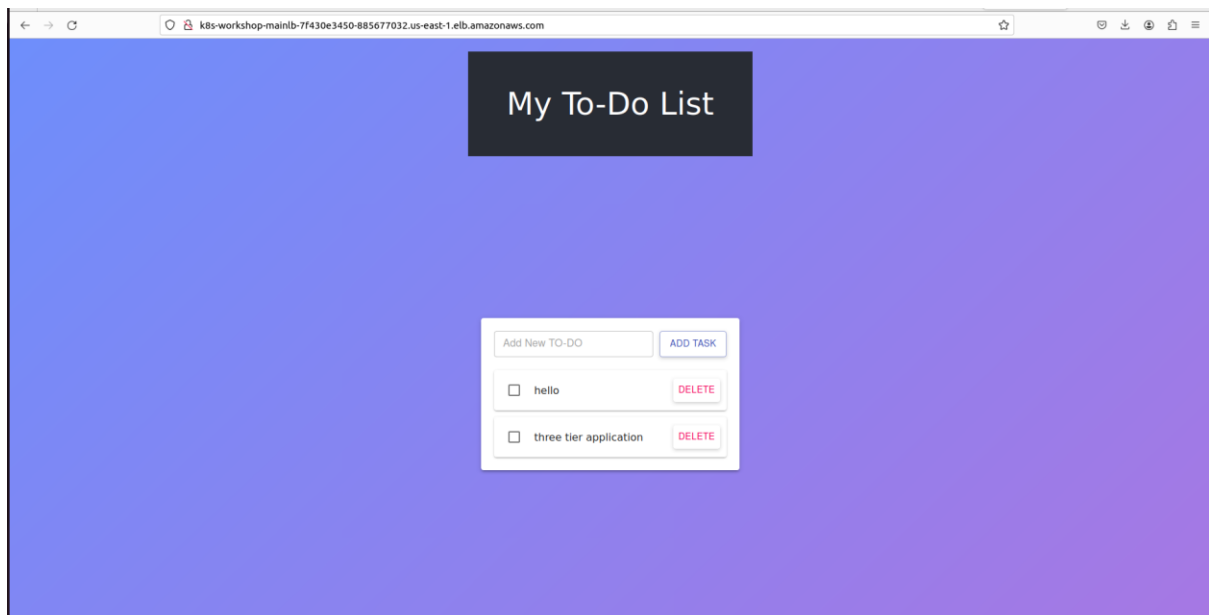
```
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests$
```

frontend-deployment.yaml

```
containers:
- name: frontend
  image: public.ecr.aws/c8x9f9q0/three-tier-frontend:latest
  imagePullPolicy: Always
  env:
    - name: REACT_APP_BACKEND_URL
      value: "http://k8s-workshop-mainlb-7f430e3450-885677032.us-east-1.elb.amazonaws.com/api/tasks"
  ports:
    - containerPort: 3000
```

full_stack_lb.yaml

```
spec:
  ingressClassName: alb
  rules:
    - host: k8s-workshop-mainlb-7f430e3450-885677032.us-east-1.elb.amazonaws.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
```



Step 14 : Check the data in the Mongo database

```

ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests$ kubectl exec -it mongodb-7f58c5f5d9-7cstv -n workshop -- /bin/sh
# ls
bin boot data dev docker-entrypoint-initdb.d etc home js-yaml.js lib lib64 media mnt opt proc root run/sbin srv sys tmp usr var
# mongo
MongoDB shell version v4.4.6
connecting to: mongodb://127.0.0.1:27017/?compressors=disabled&gssapiServiceName=mongodb
Implicit session: session { "id" : UUID("677f91bc-7b00-4782-b133-5d334049a035") }
MongoDB server version: 4.4.6
Welcome to the MongoDB shell.
For interactive help, type "help".
For more comprehensive documentation, see
  https://docs.mongodb.com/
Questions? Try the MongoDB Developer Community Forums
  https://community.mongodb.com
---
The server generated these startup warnings when booting:
  2024-01-31T03:32:18.309+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
  2024-01-31T03:32:18.309+00:00: You are running this process as the root user, which is not recommended
---
---
  Enable MongoDB's free cloud-based monitoring service, which will then receive and display
  metrics about your deployment (disk utilization, CPU, operation statistics, etc).

  The monitoring data will be available on a MongoDB website with a unique URL accessible to you
  and anyone you share the URL with. MongoDB may use this information to make product
  improvements and to suggest MongoDB products and deployment options to you.

  To enable free monitoring, run the following command: db.enableFreeMonitoring()
  To permanently disable this reminder, run the following command: db.disableFreeMonitoring()
---
> show dbs
admin    0.000GB
config  0.000GB
local    0.000GB
todo     0.000GB
> use todo
switched to db todo
> db.tasks.find()
{ "_id" : ObjectId("65ba79d7aad6876077bde532"), "completed" : false, "task" : "hello", "__v" : 0 }
{ "_id" : ObjectId("65ba7a08aad687231cbde534"), "completed" : true, "task" : "three tier application", "__v" : 0 }
> exit
bye
# exit
ubuntu@ip-172-31-82-137:~/ThreeTierApp/k8s_manifests$

```

#kubectl exec -it mongodb-7f58c5f5d9-7cstv -n workshop -- /bin/sh

This command uses `kubectl exec` to execute a command in a running container (`-it` for interactive mode) named `mongodb-7f58c5f5d9-7cstv` in the `workshop` namespace. The command to be executed is `/bin/sh`, which opens a shell in the container.

#mongo

Once inside the container's shell, the `mongo` command is used to start the MongoDB shell, which is an interactive JavaScript interface to MongoDB.

#show dbs

In the MongoDB shell, `show dbs` is a command that displays a list of all existing databases.

#use todo

The `use` command is used to switch to or create a new database. In this case, it switches to a database named `todo`.

#db.tasks.find()

After switching to the `todo` database, the `db.tasks.find()` command is used to retrieve all documents from the `tasks` collection within the `todo` database.

#exit

This command is used to exit the MongoDB shell.

Step 15 : Clean or delete the EKS cluster

`eksctl delete cluster --name three-tier-cluster --region us-east-1`